AI Assistant Coding

Assignment-11.3

B.Sanjana

2303A51455

Batch-07

Task 1: Smart Contact Manager (Arrays s Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamic memory allocation.
3. Implement the following operations in both approaches:

o Add a contact

o Search for a contact

o Delete a contact

4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:

o Insertion efficiency

o Deletion efficiency

Expected Outcome

• Two working implementations (array-based and linked-list-based).

• A brief comparison explaining performance differences.

Task 1: Smart Contact Manager (Arrays & Linked Lists)

1. Array-Based Implementation

[1]
0s

```python
class ContactManagerArray:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        self.contacts.append((name, phone))

    def search_contact(self, name):
        for contact in self.contacts:
            if contact[0] == name:
                return contact
        return "Not Found"

    def delete_contact(self, name):
        for contact in self.contacts:
            if contact[0] == name:
                self.contacts.remove(contact)
                return "Deleted"
        return "Not Found"
```

[3]
0s

```python
# Array Testing
cm = ContactManagerArray()
cm.add_contact("Ram", "12345")
cm.add_contact("Sita", "67890")

print(cm.search_contact("Ram"))
print(cm.delete_contact("Ram"))
print(cm.search_contact("Ram"))

# Linked List Testing
ll = ContactManagerLinkedList()
ll.add_contact("Ram", "12345")
ll.add_contact("Sita", "67890")

print(ll.search_contact("Sita"))
print(ll.delete_contact("Sita"))
print(ll.search_contact("Sita"))
```

```
('Ram', '12345')
Deleted
Not Found
('Sita', '67890')
Deleted
Not Found
```

2. Linked List Implementation

[2]
0s

```python
class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None

class ContactManagerLinkedList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        new_node.next = self.head
        self.head = new_node

    def search_contact(self, name):
        temp = self.head
        while temp:
            if temp.name == name:
                return (temp.name, temp.phone)
            temp = temp.next
        return "Not Found"

    def delete_contact(self, name):
        temp = self.head
        prev = None
```

```python
        return "Not Found"

    def delete_contact(self, name):
        temp = self.head
        prev = None

        while temp:
            if temp.name == name:
                if prev:
                    prev.next = temp.next
                else:
                    self.head = temp.next
                return "Deleted"
            prev = temp
            temp = temp.next
        return "Not Found"
```

Comparison:

Feature	Array	Linked List
Insertion	Fast (O(1))	Fast (O(1))
Deletion	Slow (O(n))	Faster (O(n), no shifting)
Memory	Fixed/contiguous	Dynamic

Observation:

1. The array-based implementation was simple and easy to use but required shifting elements during deletion, which increased time complexity.

2. Searching for a contact in both array and linked list required linear traversal (O(n)).

3. The linked list provided dynamic memory allocation and avoided shifting during deletion.

4. Insertion in the linked list was efficient as elements were added at the beginning (O(1)).

5. Overall, linked list performed better for frequent insertions and deletions compared to arrays.

Task 2: Library Book Search System (Queues s Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit

requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.

2. Extend the system to a Priority Queue, prioritizing faculty requests.

3. Use GitHub Copilot to assist in generating:

o enqueue() method

o dequeue() method

4. Test the system with a mix of student and faculty requests.

Expected Outcome

• Working queue and priority queue implementations.

• Correct prioritization of faculty requests.

Task 2: Library Book Search System (Queues & Priority Queues)

1. Queue Implementation (FIFO)

[5]
✓ 0s

```python
from collections import deque

class BookQueue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, request):
        self.queue.append(request)

    def dequeue(self):
        if self.queue:
            return self.queue.popleft()
        return "Empty Queue"
```

2. Priority Queue (Faculty Priority)

[6]
✓ 0s

```python
import heapq

class PriorityBookQueue:
    def __init__(self):
        self.queue = []
```

[6]
✓ 0s

```python
        self.queue = []

    def enqueue(self, name, role):
        priority = 0 if role == "faculty" else 1
        heapq.heappush(self.queue, (priority, name))

    def dequeue(self):
        if self.queue:
            return heapq.heappop(self.queue)
        return "Empty"
```

[14]
✓ 0s

```python
# Normal Queue
q = BookQueue()
q.enqueue("Student1")
q.enqueue("Student2")

print(q.dequeue())
print(q.dequeue())

# Priority Queue
pq = PriorityBookQueue()
pq.enqueue("Student1", "student")
pq.enqueue("Faculty1", "faculty")
pq.enqueue("Student2", "student")

print(pq.dequeue())
print(pq.dequeue())
print(pq.dequeue())
```

[14]
✓ 0s


```python
print(q.dequeue())
print(q.dequeue())

# Priority Queue
pq = PriorityBookQueue()
pq.enqueue("Student1", "student")
pq.enqueue("Faculty1", "faculty")
pq.enqueue("Student2", "student")

print(pq.dequeue())
print(pq.dequeue())
print(pq.dequeue())
```


```
Student1
Student2
(0, 'Faculty1')
(1, 'Student1')
(1, 'Student2')
```

Observation:

- The queue followed FIFO order, ensuring fairness in processing requests.
- The priority queue successfully prioritized faculty requests over student requests.
- Heap-based priority queue efficiently managed ordering with O(log n) operations.
- The system handled mixed inputs correctly, always serving higher-priority requests first.
- This demonstrated the importance of choosing appropriate data structures based on requirements.

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students

and staff. While tickets are received sequentially, issue escalation follows a

Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.

2. Provide the following operations:

o push(ticket)

o pop()

o peek()

3. Simulate at least five tickets being raised and resolved.

4. Use GitHub Copilot to suggest additional stack operations such as:

o Checking whether the stack is empty

o Checking whether the stack is full (if applicable)

Expected Outcome

• Functional stack-based ticket management system.

• Clear demonstration of LIFO behavior.

Task 3: Emergency Help Desk (Stack Implementation)

Stack Implementation

[8]
0s

```python
class Stack:
    def __init__(self):
        self.stack = []
        self.limit = 5

    def push(self, ticket):
        if len(self.stack) < self.limit:
            self.stack.append(ticket)
        else:
            print("Stack Full")

    def pop(self):
        if self.stack:
            return self.stack.pop()
        return "Empty"

    def peek(self):
        if self.stack:
            return self.stack[-1]
        return "Empty"

    def is_empty(self):
        return len(self.stack) == 0
```

[8]
0s

```python
        return len(self.stack) == 0

    def is_full(self):
        return len(self.stack) == self.limit
```

[9]
0s

```python
s = Stack()
s.push("T1")
s.push("T2")
s.push("T3")
s.push("T4")
s.push("T5")

print(s.pop())
print(s.peek())
```

```
T5
T4
```

Task 4: Hash Table

[10]
0s

```python
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return hash(key) % self.size
```

Observation:

- The stack followed LIFO behavior, where the most recent ticket was resolved first.
- Push and pop operations were efficient (O(1)).
- The simulation clearly showed how recently added tickets are handled first.
- Additional operations like is_empty() and is_full() improved reliability.
- Stack is suitable for scenarios involving undo operations or backtracking.

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

• Insert

• Search

• Delete

Starter Code

class HashTable:

pass

Expected Outcome

• Collision handling using chaining

• Well-commented methods

Task 4: Hash Table

[10]
0s

```python
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        self.table[index].append((key, value))

    def search(self, key):
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return "Not Found"

    def delete(self, key):
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return "Deleted"
        return "Not Found"
```

[13]
0s

```python
ht = HashTable()

ht.insert("Ram", "12345")
ht.insert("Sita", "67890")

print(ht.search("Ram"))
print(ht.delete("Ram"))
print(ht.search("Ram"))
```

```
12345
Deleted
Not Found
```

Observation:

- Hash table provided fast insertion, deletion, and search operations (average O(1)).
- Collision handling using chaining worked effectively by storing multiple elements in the same index.
- Performance depends on the quality of the hash function.
- As the number of collisions increased, performance slightly decreased.
- Hash tables are ideal for quick data retrieval using keys.

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

- Mapping table: Feature → Data Structure → Justification
- One fully working Python implementation

Task 5: Real-Time Application Challenge

[11]
0s

```python
from collections import deque

class CafeteriaQueue:
    def __init__(self):
        self.orders = deque()

    def place_order(self, order):
        self.orders.append(order)

    def serve_order(self):
        if self.orders:
            return self.orders.popleft()
        return "No Orders"

cq = CafeteriaQueue()
cq.place_order("Burger")
cq.place_order("Pizza")

print(cq.serve_order())
```

```
Burger
```

```
[12]
✓ 0s
cq = CafeteriaQueue()

cq.place_order("Burger")
cq.place_order("Pizza")

print(cq.serve_order())
print(cq.serve_order())
print(cq.serve_order())
```

```
Burger
Pizza
No Orders
```

Observation:

- Different features required different data structures based on functionality.
- Queue was best suited for order processing and request handling.
- Hash table provided efficient lookup for attendance tracking.
- Priority queue helped manage tasks based on importance.
- Selecting the correct data structure improved efficiency and system performance.